PROJECT_QLE

Deep Code Documentation

Every line explained — with the reasoning behind each decision

Eng. Qusai Alnuaimat · Dr. Lutfi Dugdug

Waha Oil Company — Exploration Department

Version 1.0 | May 2026 | Confidential

Table of Contents

1. Project Philosophy and Design Goals

Project_QLE was built by Eng. Qusai Alnuaimat and Dr. Lutfi Dugdug at Waha Oil Company's Exploration Department to address a specific operational problem: well log interpretation was being done in disconnected tools — spreadsheets for petrophysics, separate scripts for statistics, manual PowerPoint slides for reports. Every exploration study required hours of copy-paste work, and results were hard to reproduce.

The platform is designed around five principles that explain many code decisions you will see throughout this documentation:

- **One language, one environment.** Python + Streamlit means the same code runs analysis and displays results. No Excel exports, no separate visualisation tools.
- **Libya-first calibration.** Every petrophysical default, every AI prompt, every formation database entry is calibrated for Libyan geology — not generic textbook values.
- **Fail gracefully.** Every parser, every calculation, every AI call is wrapped in try/except. One bad well file does not crash the entire project.
- **Metric internally, US field units for display.** All mathematics uses SI units (metres, g/cc) because the equations are derived in SI. Display converts to feet, psi, etc. because that is the NOC field standard.
- **Explicit over implicit.** Every parameter has a name and a default. No magic numbers buried inside formulas without explanation.

2. units.py — US Field Unit System

project_QLE/units.py

2.1 Why Store Metric but Display US Units?

All standard petrophysical equations (Archie, Simandoux, Eaton, Wyllie) are published and taught in metric SI units. The textbooks, the software manuals, and the equations themselves use metres, g/cc, and ohm-metres. Switching the internal maths to feet would require converting every constant in every equation — error-prone and unverifiable.

However, NOC Libya and Waha Oil Company field operations use US field units: depths in feet, pressures in psi, mud weight in lb/gal. The unit conversion therefore happens in only one place — at the display layer — rather than scattered through every formula.

> ***Why this matters:*** *Centralising conversions in units.py means if the conversion factor ever needs updating (e.g. a more precise M_TO_FT value), you change it in one place and it propagates everywhere automatically.*

2.2 Every Constant and Function Explained

```
M_TO_FT   = 3.28084   # The exact NIST definition: 1 metre = 3.28084 feet
FT_TO_M   = 1.0 / M_TO_FT   # Inverse — computed once, not typed repeatedly
```

Explanation: We write 1.0/M_TO_FT rather than 0.3048 to guarantee the inverse is numerically consistent with the forward conversion. If you convert 1 ft to m and back, you should get exactly 1 ft — not 0.9999999 due to floating-point rounding.

```
BAR_TO_PSI = 14.5038   # 1 bar = 14.5038 psi (exact NIST value)
G_CC_TO_LB_GAL = 8.33  # mud weight: 1 g/cc ≈ 8.33 ppg (pounds per gallon)
M3_TO_BBL  = 6.28981   # 1 cubic metre = 6.28981 US barrels
```

Explanation: G_CC_TO_LB_GAL converts formation water or mud density from the metric drilling measurement (g/cc) to US field mud weight (ppg). A 1.05 g/cc formation water equals 1.05 × 8.33 = 8.75 ppg.

```
def m_to_ft(val):
    if val is None:
        return None    # Propagate None instead of crashing on None * 3.28084
    return val * M_TO_FT   # Works on scalars AND numpy arrays
```

Explanation: The None check is critical. Many well header fields (like start_depth) might be None if the LAS file header was incomplete. Without this check, 'None * 3.28084' raises a TypeError that would crash the entire display layer.

```
def fmt_depth(m: float) -> str:
    if m is None:
        return 'N/A'           # Human-readable missing value
    return f'{m_to_ft(m):.0f} ft'  # Zero decimal places — depths in ft need no decimals
```

Explanation: We use :.0f (zero decimal places) for depth in feet because the LAS file depth step is 0.5m = 1.64 ft. Showing sub-foot precision (e.g. 6561.7 ft) implies accuracy we don't have. 6562 ft is the correct display.

2.3 UNIT_LABELS Dictionary

```
UNIT_LABELS = {
    'DEPTH':          'ft',
    'GR':             'GAPI',   # Gamma Ray in API units — a US convention
    'RHOB':           'g/cc',   # Stays metric — no US equivalent in common use
    'NPHI':           'v/v',    # Volume fraction — dimensionless
    'RT':             'Ω·m',    # Resistivity — always ohm-metres internationally
    'DT':             'µs/ft',  # Sonic in microseconds per foot — already US
```

```
    'PORE_PRESS_PSI': 'psi',   # Pressure displayed in psi
}
```

Explanation: Notice that RHOB stays in g/cc — this is because 'g/cc' is universally used by all logging tools, including US service companies. There is no US field equivalent that is more familiar to geologists. Similarly, DT is already in µs/ft — sonic tools worldwide report in feet regardless of the depth convention.

3. core/models.py — All Data Structures

The language every module speaks

3.1 Why Python dataclasses Instead of Plain Dicts?

A Python dict like {'well_name': 'SARIR-1', 'basin': 'SIRTE'} has no enforcement: you can mis-spell 'well_nmae' and Python will never tell you. A dataclass defines the exact field names and types once, and any typo becomes an AttributeError immediately. It also provides auto-generated __repr__ for debugging and type hints that IDEs can check.

Why this matters: *In a geoscience project where data flows through ten different modules — parsers, petrophysics, facies, AI — using typed dataclasses means a broken data structure fails loudly at the module boundary, not silently deep inside a calculation where the original source is hard to trace.*

3.2 str Enum — Why Each Enum Is Also a String

```
class Facies(str, Enum):      # Inherits from BOTH str AND Enum
    SANDSTONE = 'Sandstone'   # The value IS the string
    SHALE     = 'Shale'
```

Explanation: By inheriting from str, we get Facies.SANDSTONE == 'Sandstone' is True. This means facies labels can be stored directly in a Pandas DataFrame column or a SQLite TEXT column without any serialisation step. When you read a label back from the database as the string 'Sandstone', Facies('Sandstone') will reconstruct the enum correctly.

3.3 FileType, CurveType, Facies, FluidType — Purpose of Each

FileType	Used by file_parser.py to choose which parser function to call. UNKNOWN is the safe fallback that returns an empty ParsedFile stub rather than raising an error.
CurveType	Standardises curve names from different LAS vendors. Both 'ILD' and 'LLD' from different service companies map to CurveType.RT (true resistivity).
Facies	Eight lithological classes. Coal and Salt are included even though they are rare in Libya — encountering them in a thin interbedded sequence should not cause an enum ValueError.
FluidType	DRY is the fifth option — used when a zone meets net reservoir cutoffs but has no fluid signature (e.g. tight gas shows no resistivity contrast).

3.4 ParsedFile — The Universal File Container

```
@dataclass
class ParsedFile:
    source_path : Path              # Always stored — needed for reload
    file_type   : FileType          # Enum, not string — prevents typos
    raw_text    : Optional[str] = None   # For PDF, DOCX, XML
    dataframe   : Optional[Any] = None   # For CSV, LAS, DOCX tables
    metadata    : Dict[str, Any] = field(default_factory=dict)  # file-specific
    images      : List[bytes]    = field(default_factory=list)  # PDF images
    extra       : Dict[str, Any] = field(default_factory=dict)  # overflow
    parsed_at   : datetime       = field(default_factory=datetime.utcnow)
```

Explanation: field(default_factory=dict) — NOT field(default={}). In Python, using a mutable default (like {}) in a dataclass means ALL instances share the SAME dict object. Changes to one ParsedFile's metadata would corrupt every other ParsedFile. default_factory creates a fresh dict for each instance.

■ **Important:** This is one of the most common Python bugs. If you ever see strange data appearing in the wrong objects, check whether a mutable default was used instead of default_factory.

3.5 WellHeader — Every Field Explained

well_name	Free text from the LAS WELL section. Defaults to 'UNKNOWN' rather than None — prevents NoneType errors in string formatting.
uwi	Unique Well Identifier — the 16-character industry standard ID. Optional because Libyan LAS files often omit it.
basin	Defaults to 'SIRTE' — the most common Libyan basin. Set by the app when a user selects a basin before uploading.
kb_elev	Kelly Bushing elevation in metres. Needed to convert measured depth (MD) to true vertical depth (TVD). Often absent.
null_value	-999.25 is the LAS standard for missing data. The LAS spec chose this because it is unlikely to occur as a real measurement.

3.6 WellCurve and the .array Property

```
@dataclass
class WellCurve:
    data : List[float] = field(default_factory=list)  # Stored as Python list

    @property
    def array(self) -> np.ndarray:
        return np.array(self.data, dtype=float)   # Converts on demand
```

Explanation: We store data as a Python list (not a numpy array) because Python lists serialise cleanly to JSON, SQLite TEXT, and pickle — all of which are used for persistence. NumPy arrays require special handling in all three cases. The .array property converts to NumPy on demand, paying the small conversion cost only when calculations actually need the array.

3.7 WellLog.get_depth() — Order Matters

```
def get_depth(self) -> Optional[np.ndarray]:
    for key in ('DEPT', 'MD', 'DEPTH'):   # Priority order
        if key in self.curves:
            return self.curves[key].array
    return None
```

Explanation: The order DEPT → MD → DEPTH reflects real LAS file conventions. Most LAS files use 'DEPT' (the original LAS spec mnemonic). Some service companies use 'MD' (measured depth). Others write 'DEPTH'. By trying all three in order, the parser handles all variants without the user needing to know which convention their vendor used.

4. core/libya_geology.py — Basin Configuration

Why every number is what it is

4.1 Why All Parameters Live in One File

Petrophysical analysis is not universal — it is calibrated. The same sonic log value means different porosities in a Sirte Basin limestone (rho_matrix = 2.71 g/cc) versus a Murzuq Basin sandstone (rho_matrix = 2.65 g/cc). If these values were scattered across five different analysis files, updating a calibration after a core plug study would require finding and changing five places. With a central file, one change propagates everywhere automatically.

4.2 Every Parameter With Its Physical Meaning

gr_clean	The gamma ray reading in a completely shale-free rock (GAPI). In Sirte carbonates this is 12 GAPI — very low because limestone contains almost no radioactive elements. In Ghadames sandstones it is 22 GAPI because quartz sands pick up a little potassium from feldspar and mica. The difference between gr_clean and gr_shale is the range over which Vshale is calculated.
gr_shale	The gamma ray reading in 100% shale. Sirte = 90 GAPI. Ghadames = 105 GAPI. Murzuq = 92 GAPI. These differ because different basins have different clay mineral compositions — illite and smectite have different radioactivity.
rho_matrix	The density of the rock grains themselves (not the whole rock). Limestone = 2.71 g/cc (calcite mineral density). Quartz sandstone = 2.65 g/cc. This is the most critical number for density porosity. Wrong rho_matrix shifts every porosity calculation in the well by a constant offset.
rho_fluid	The density of the fluid in the borehole mud column (not reservoir fluid). Fresh water mud = 1.00 g/cc. Slightly saline mud = 1.05 g/cc (Sirte wells). Salt-saturated mud = 1.10 g/cc. The density log measures bulk density, which is rho_matrix × (1-phi) + rho_fluid × phi. Wrong rho_fluid causes systematic porosity errors.
rw	Formation water resistivity in ohm-metres. This is the most sensitive Archie parameter — doubling rw doubles the computed water saturation. Sirte Paleocene = 0.025 ohm-m (very saline, high NaCl). Ghadames = 0.042 ohm-m (less saline). These values come from formation water samples collected during DST testing.
rsh	Shale resistivity in ohm-metres. Used in the Simandoux equation. Lower rsh means the shale conducts electricity more efficiently, which artificially lowers apparent resistivity in shaly sands. Ignoring this (using Archie only) would overestimate Sw in shaly intervals.
vsh_method	Which Gamma Ray to Vshale transform to apply. Larionov Young = for Mesozoic and older rocks (Sirte, Murzuq). Larionov Old = for Tertiary rocks (Ghadames Acacus). The Young transform gives lower Vshale at intermediate GR values, which is correct for older, more compacted shales.
overburden_gradient	Rate of total overburden pressure increase with depth (psi/ft). This is the weight of all rock above divided by depth. Sirte = 0.95 psi/ft (slightly lighter than global average of 1.0 psi/ft because Sirte carbonates are less dense than clastic-dominant sequences).
hydrostatic_gradient	Rate of pore water pressure increase if the formation were normally pressured. Fresh water = 0.433 psi/ft. Marine (offshore) = 0.465 psi/ft. This is the baseline against which overpressure is measured.

normal_dt_surface	The expected sonic travel time at the surface (depth = 0) in microseconds per foot. As sediment compacts with burial, DT decreases exponentially. Sirte = 130 µs/ft at surface (soft, porous near-surface carbonates). Ghadames = 120 µs/ft (harder surface formations). This anchors the normal compaction trend for Eaton pore pressure calculation.
normal_dt_exp	The exponential decay constant for the normal compaction trend. Negative value: -0.00020 means DT decreases 0.02% per foot of depth. At 10,000 ft: dt_normal = 130 × e^(–0.00020 × 10000) = 130 × 0.135 = 17.6 µs/ft. Calibrated from offset well data across each basin.

4.3 get_basin_defaults() — The Fallback Design

```
def get_basin_defaults(basin: str) -> Dict:
    key = basin.strip().upper()          # Normalise: 'sirte' → 'SIRTE'
    return dict(_BASIN_DEFAULTS.get(key, _BASIN_DEFAULTS['SIRTE']))
    #                          ↑ fallback ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Explanation: The .get() with a SIRTE fallback means that if someone passes 'KUFRA' (a frontier basin with few wells and unknown parameters), the system does not crash — it uses Sirte defaults, which are reasonable for Libyan carbonates. The dict() call creates a COPY of the defaults so the caller can safely mutate it (e.g. override one parameter) without corrupting the master defaults dictionary.

5. parsers/las_parser.py — LAS File Reader

5.1 What Is a LAS File?

LAS (Log ASCII Standard) was defined by the Canadian Well Logging Society in 1989. It is a plain-text file with four sections: VERSION (LAS 1.2/2.0/3.0), WELL (header metadata), CURVE (curve name definitions), and ~A (the actual data as space-delimited numbers). A 3000-metre well at 0.1m sampling with 10 curves produces a ~300,000-line file.

5.2 lasio — Why This Library

lasio is the industry-standard Python LAS parser, used by major operators and service companies. It handles LAS 1.2, 2.0, and 3.0 automatically, parses the WELL section into HeaderItem objects, builds a DataFrame with las.df(), and handles wrapped lines and non-standard encodings. Writing our own LAS parser would take months and introduce edge-case bugs that lasio has already solved.

5.3 _MNEMONIC_MAP — Why This Is Necessary

```
_MNEMONIC_MAP = {
    'GR'   : CurveType.GR,
    'ILD'  : CurveType.RT,   # Induction Deep → True Resistivity
    'LLD'  : CurveType.RT,   # Laterolog Deep → True Resistivity
    'RHOB' : CurveType.RHOB,
    'RHOZ' : CurveType.RHOB,  # Schlumberger's name for bulk density
    'NPHI' : CurveType.NPHI,
    'TNPH' : CurveType.NPHI,  # Baker Hughes thermal neutron
    'DT'   : CurveType.DT,
    'DTC'  : CurveType.DT,   # Compressional DT
    'DTCO' : CurveType.DT,   # Schlumberger's name for P-wave sonic
}
```

Explanation: Each logging service company (Schlumberger, Halliburton, Baker Hughes, Core Labs) uses different mnemonic names for the same measurement. RHOB (Halliburton) and RHOZ (Schlumberger) both measure bulk density. Without this map, petrophysics.py would have to ask for both names every time. The map normalises everything to one canonical name so downstream code only knows one name per curve type.

5.4 Null Value Handling — Why -999.25

```
data = np.where(
    np.isclose(curve_item.data, null_val, atol=0.01),  # Within 0.01 of null
    np.nan,           # Replace with NaN
    curve_item.data,  # Otherwise keep original
).tolist()
```

Explanation: We use np.isclose() with atol=0.01 rather than exact equality (==) because floating-point storage can introduce tiny rounding errors: -999.25 stored as a 32-bit float may read back as -999.2500122. An exact equality test would miss it. NaN is Python/NumPy's native missing value — all arithmetic with NaN returns NaN, and np.nanmean() / dropna() can skip NaN rows automatically.

5.5 Building the Convenience DataFrame

```
df = las.df().rename_axis('DEPTH').reset_index()
# las.df() returns a DataFrame indexed by depth
# rename_axis gives the index column a name
# reset_index() promotes the depth index to a regular column
df.columns = [c.upper() for c in df.columns]  # Uppercase all column names
```

```
df.replace(null_val, np.nan, inplace=True)     # Final null cleanup
```

Explanation: Uppercasing column names is critical: some LAS files write 'gr' (lowercase) and others write 'GR'. Without uppercasing, the column lookup 'GR' would miss 'gr', causing silent NaN results in petrophysics. This single line prevents a class of bugs that are very hard to debug.

6. parsers/file_parser.py — Universal Parser Dispatcher

6.1 The Dispatch Table Pattern

```
_PARSERS = {
    FileType.PDF  : _parse_pdf,
    FileType.DOCX : _parse_docx,
    FileType.XML  : _parse_xml,
    FileType.JPG  : _parse_image,
    FileType.CSV  : _parse_csv,
}

def parse_file(path) -> ParsedFile:
    file_type = detect_file_type(path)     # .pdf → FileType.PDF
    parser = _PARSERS.get(file_type)        # Look up the function
    if parser is None:
        return ParsedFile(source_path=path, file_type=file_type)  # Stub
    return parser(path)                     # Call the right parser
```

Explanation: A dispatch table (dict mapping type → function) is cleaner than a long if/elif chain. Adding support for a new file format (e.g. .dlis seismic) requires adding one entry to _PARSERS and one function — nothing else changes. With an if/elif chain, you'd have to edit the main logic flow.

6.2 Each Parser Function

_parse_pdf	Uses PyMuPDF (fitz). page.get_text() extracts text from each page. page.get_images(full=True) finds all embedded images. extract_image(xref) returns the raw image bytes. All page texts are joined with newlines.
_parse_docx	Uses python-docx. doc.paragraphs gives all text blocks. doc.tables gives all tables as grids of cells. Tables are converted to DataFrames — only the first table becomes the primary DataFrame; all are stored in extra['all_tables'].
_parse_csv	pd.read_csv with low_memory=False prevents Pandas from using the first 100 rows to guess types (which misses columns where the first rows are text headers). df.apply(pd.to_numeric, errors='ignore') converts number-like string columns to float while leaving true text columns untouched.

7. parsers/intelligent_parser.py — AI Document Reader

Gemini extracts petrophysical data from PDFs/CSVs

7.1 Purpose and Design

Geoscientists routinely work with PDFs: well completion reports, mudlog printouts, core analysis reports, DST summaries. These contain critical data (formation names, depths, flow rates, pressure readings) trapped in unstructured text. Before intelligent_parser.py, someone had to manually read each document and type the numbers into the app. Now the app reads the document and populates the fields automatically.

The design uses two layers: rule-based extraction (fast, always works, no API key needed) plus Gemini AI enhancement (slower, deeper, requires API key). If Gemini is not available, the rule-based layer still extracts numbers, well names, and formation tops using regular expressions.

7.2 DocumentExtraction Dataclass

```
@dataclass
class DocumentExtraction:
    source_file     : str              # Track provenance
    document_type   : str = 'unknown'  # 'well_report', 'dst_report', etc.
    well_names      : List[str] = field(default_factory=list)
    formation_tops  : List[Dict] = field(default_factory=list)
    dst_tests       : List[Dict] = field(default_factory=list)
    petro_dataframe : Optional[pd.DataFrame] = None
    key_values      : Dict[str, Any] = field(default_factory=dict)
    ai_summary      : str = ''
    warnings        : List[str] = field(default_factory=list)
```

Explanation: The separation into formation_tops (list of dicts) and dst_tests (list of dicts) mirrors the app's internal data structure for those pages. The extraction result can be directly passed to the Formation Tops page and DST Tests page without transformation — the keys in each dict match exactly what those pages expect.

7.3 Column Auto-Mapping (_COLUMN_SYNONYMS)

```
_COLUMN_SYNONYMS = {
    'GR': ['gr', 'gamma ray', 'gamma_ray', 'gammaray', 'gapi'],
    'RHOB': ['rhob', 'density', 'bulk density', 'rhoz', 'den'],
    'PHIE': ['phie', 'porosity', 'eff porosity', 'phi_e', 'phi'],
    ...
}

def _map_column(col_name: str) -> str:
    lower = col_name.lower().strip().replace(' ', '_').replace('-', '_')
    for canonical, synonyms in _COLUMN_SYNONYMS.items():
        if lower == syn or lower.startswith(syn):  # Prefix match
            return canonical
    return col_name.upper()  # Unknown → uppercase as-is
```

Explanation: The prefix match (startswith) handles columns like 'gr_core' or 'porosity_log' that embed the standard mnemonic within a longer name. The fallback to uppercase means unknown columns are preserved rather than silently lost — the geologist sees 'FORMATION_DEPTH' in the mapped DataFrame and can decide how to use it.

8. analysis/petrophysics.py — Full Line-by-Line

The most critical analysis module

8.1 clamp() and _safe_get()

```
def clamp(a: np.ndarray, lo=0.0, hi=1.0) -> np.ndarray:
    return np.clip(a, lo, hi)
# Called after EVERY saturation or fraction calculation.
# Prevents physically impossible values like Sw = 1.3 or phi = -0.05
# that arise from noisy tool data or equation overshoot.
```

Explanation: np.clip is a single vectorised operation — it clamps an entire array in one call without Python-level loops. This is important because a 3000m well at 0.1m sampling has 30,000 depth points. Looping would be 30,000× slower.

```
def _safe_get(well: WellLog, *mnemonics: str) -> Optional[np.ndarray]:
    for m in mnemonics:           # Try each name in order
        v = well.get_curve(m)
        if v is not None:         # Return first match found
            return v
    return None                   # None if no variant found
```

Explanation: The *mnemonics variadic parameter allows passing multiple alternative names: _safe_get(w, 'RT', 'ILD', 'LLD', 'MSFL') tries all four resistivity mnemonics in priority order. RT (true resistivity) is most accurate; MSFL (micro-spherically focused log) measures flushed zone and is least accurate — so it comes last.

8.2 vshale_gr() — Five Equations Explained

```
igr = clamp((gr - gr_clean) / (gr_shale - gr_clean + 1e-9))
# IGR = Gamma Ray Index, normalised 0 to 1
# 1e-9 added to denominator prevents division by zero if gr_clean == gr_shale
```

IGR is the linear normalisation of GR between the clean (0) and shale (1) endpoints. It is the INPUT to all Vshale transforms. Then:

Linear Vsh = IGR	The simplest formula. Assumes a linear relationship between GR and clay content. Always gives the HIGHEST Vshale of the five methods — it is the most conservative (pessimistic for the reservoir).
Larionov Old 0.33 × (2^(2×IGR) – 1)	Calibrated from Tertiary (young) rocks in the Gulf of Mexico. The exponential term makes Vshale lower than linear at high IGR, reflecting that young shales have higher radioactivity due to organic content.
Larionov Young 0.083 × (2^(3.7×IGR) – 1)	Calibrated for Mesozoic and older rocks — including Sirte Basin carbonates and Paleozoic sandstones. Gives even lower Vshale than Larionov Old. Used for Libya because the formations are Cretaceous–Ordovician age.
Clavier 1.7 – √(3.38 – (IGR+0.7)²)	An empirical equation from a different dataset. Used as a cross-check when the Larionov result seems too low. The square root term compresses the Vshale scale at intermediate IGR values.
Stieber IGR / (3 – 2×IGR)	Originally developed for dispersed shale in sands. The denominator grows with IGR, compressing Vshale at high GR values.

8.3 porosity_density() — The Most Important Formula

```
def porosity_density(rhob, rho_matrix=2.71, rho_fluid=1.05):
    return clamp((rho_matrix - rhob) / (rho_matrix - rho_fluid + 1e-9))
#                              ↑
# rho_matrix=2.71 → Sirte carbonates (calcite density)
# rho_fluid=1.05  → slightly saline Sirte mud filtrate
```

Explanation: This is the bulk density equation: rhob = rho_matrix×(1–phi) + rho_fluid×phi. Solving for phi: phi = (rho_matrix – rhob) / (rho_matrix – rho_fluid). The formula is exact for a two-component system (matrix + fluid). In reality, gas in the pores reduces rhob further than predicted (gas effect), and heavy minerals increase it. The clamping to [0,1] corrects these edge cases.

■ **Important:** Using rho_matrix = 2.65 (sandstone) on a Sirte carbonate well gives porosity readings about 3–5% lower than the true value. This is a common field error that can change a 'Good' reservoir to a 'Fair' one. Always verify rho_matrix before interpreting porosity results.

8.4 porosity_neutron_density() — The Gas Detector

```
def porosity_neutron_density(nphi, phid):
    return clamp(np.sqrt((nphi**2 + phid**2) / 2))
# Root-mean-square average of the two porosity measurements
```

Explanation: The neutron log (NPHI) measures hydrogen concentration — gas has much lower hydrogen density than oil or water, so NPHI reads LOW in gas. The density log reads LOW because gas is less dense than oil/water. The crossover of NPHI < PHID (gas crossover) on a standard log display is the classic gas indicator. The RMS average partially cancels the gas effect, giving a better porosity estimate in hydrocarbon-bearing intervals than either tool alone.

8.5 porosity_sonic_wyllie() — Time Average Equation

```
def porosity_sonic_wyllie(dt, dt_matrix=47.6, dt_fluid=189.0, cp=1.0):
    return clamp(((dt - dt_matrix) / (dt_fluid - dt_matrix + 1e-9)) / cp)
# dt_matrix=47.6 → Sirte limestone (47.6 µs/ft for calcite)
# dt_matrix=55.5 → sandstone (quartz)
# dt_fluid=189.0 → fresh water (borehole mud filtrate)
# cp=1.0         → compaction factor (1.0 = no correction needed)
```

Explanation: The Wyllie time-average equation models the rock as a parallel combination of matrix and fluid acoustic paths: 1/DT = (1-phi)/DT_matrix + phi/DT_fluid. Solving for phi gives the Wyllie formula above. The compaction correction cp > 1.0 is applied in soft, undercompacted sands (shallow depths) where the rock is more porous than the equation assumes.

8.6 sw_archie() — The Fundamental Saturation Equation

```
def sw_archie(rt, phi, rw=0.025, a=1.0, m=2.0, n=2.0):
    sw = ((a * rw) / (rt * phi**m + 1e-9))**(1/n)
    return clamp(sw)
# Parameters:
# rw = formation water resistivity (ohm-m) — MOST sensitive parameter
# a  = tortuosity factor (1.0 = Archie's original value)
# m  = cementation exponent (2.0 = consolidated, 1.5 = unconsolidated)
# n  = saturation exponent (2.0 = default, higher = wettability effects)
```

Explanation: The cementation exponent m encodes the connectivity of pore space. In a well-cemented carbonate (m = 2.5), the electrical path between pores is very tortuous — the rock looks more resistive than its porosity alone would suggest, which the equation corrects for by raising phi to the m power. Lower m = more connected pores = lower apparent resistivity at same Sw.

8.7 sw_simandoux() — Shaly Sand Correction

```
def sw_simandoux(rt, phi, vsh, rw=0.025, rsh=1.5, ...):
    c = phi**m / (a * rw + 1e-9)   # Clean sand conductivity term
    d = vsh / (rsh + 1e-9)          # Shale conductivity term
    sw = (c/(2*rt)) * (sqrt(1 + (2*rt*d/c)**2) - 2*rt*d/c)**(2/n)
    return clamp(sw)
```

Explanation: In a shaly sand, the clay minerals provide an additional electrical conduction path parallel to the water-filled pores. This makes the formation appear more conductive (lower Rt) than a clean sand with the same

Sw. Archie interprets this as higher water saturation — overestimating Sw and underestimating hydrocarbon saturation. Simandoux adds the shale conductivity term d = Vsh/Rsh to account for this extra path. The formula is used whenever VSHALE > 15%.

8.8 pore_pressure_eaton() — Overpressure Detection

```
depth_ft = depth_m * 3.28084      # Convert: Eaton defined in psi/ft system
if dt_normal is None:
    dt_normal = normal_dt_surface * np.exp(normal_dt_exp * depth_ft)
#   ↑ Builds normal compaction trend from surface to TD

ob = overburden_gradient * depth_ft   # Overburden pressure at each depth
hy = hydrostatic_gradient * depth_ft  # Hydrostatic pressure at each depth
ratio = dt_normal / dt_obs            # Observed/Normal DT ratio
pp = ob - (ob - hy) * ratio**eaton_exp  # Eaton equation
```

Explanation: The Eaton equation: PP = OBG – (OBG – HG) × (DT_normal/DT_obs)^n. When DT_obs = DT_normal (normal compaction), ratio = 1, and PP = OBG – (OBG – HG) × 1 = HG. Normal pressure. When DT_obs > DT_normal (undercompaction, overpressure), ratio < 1, and (ratio)^3 << 1, so PP approaches OBG. Severe overpressure. The exponent n=3 was empirically calibrated by Eaton for sonic logs.

8.9 PetrophysicsEngine.__init__() — Three-Level Parameter Override

```
def __init__(self, well, basin='SIRTE', **overrides):
    cfg = get_basin_defaults(self.basin)  # Level 1: basin defaults
    cfg.update(overrides)                  # Level 2: user overrides win
    self.gr_clean = cfg['gr_clean']        # Level 3: hardcoded fallback
    ...
    self.is_carbonate = self.rho_matrix >= 2.70  # Decide perm equation
```

Explanation: The three-level override system: basin defaults → user overrides → hardcoded fallbacks. cfg.update(overrides) means any keyword argument to PetrophysicsEngine() overrides the basin default for that parameter only. You can write PetrophysicsEngine(well, basin='SIRTE', rw=0.03) to use a custom Rw from a water sample without having to specify every other Sirte parameter. is_carbonate selects Carmen-Kozeny (carbonates) vs Timur (clastics) automatically.

9. analysis/facies.py — Lithological Classification

9.1 _build_feature_matrix() — NaN Handling Strategy

```
imputer = SimpleImputer(strategy='median')
X = imputer.fit_transform(X)
# Replace NaN with column median before clustering
# WHY median not mean: median is robust to outliers
# A single erroneous GR spike of 300 GAPI would drag the mean up
# but barely affect the median.
```

Explanation: KMeans fails completely if the input array contains NaN — it propagates through all distance calculations and all cluster assignments become NaN. The SimpleImputer with median strategy fills gaps with the most representative value for that curve, allowing clustering to proceed on partial data.

9.2 RuleBasedFacies.classify() — Decision Tree Logic

```
if g >= self.gr_shale_min:
    labels[i] = Facies.SHALE          # High GR = clay = shale
elif g <= self.gr_sand_max:
    if r >= self.rho_lim_evap:         # Dense + clean = evaporite
        labels[i] = Facies.ANHYDRITE
    elif r >= self.rho_lim_carb:       # 2.70–2.80 g/cc = carbonate
        labels[i] = Facies.DOLOMITE if r >= 2.80 else Facies.LIMESTONE
    else:                              # < 2.55 g/cc = clastic
        labels[i] = Facies.SANDSTONE
```

Explanation: The density sequence (sandstone < limestone < dolomite < anhydrite) reflects actual mineral densities: quartz = 2.65, calcite = 2.71, dolomite = 2.87, anhydrite = 2.98 g/cc. The GR filter first separates clay from non-clay, then density separates the non-clay lithologies. This two-step approach handles the most common ambiguities: tight clean sands can look like carbonates on GR alone, but density separates them.

9.3 KMeansFacies — Why StandardScaler Is Essential

```
self.scaler = StandardScaler()
Xs = self.scaler.fit_transform(X)
# StandardScaler: subtracts mean, divides by std deviation
# Before: GR ranges 0–200 GAPI, NPHI ranges 0–0.5 v/v
# After:  both have mean=0, std=1
# WHY: KMeans uses Euclidean distance. Without scaling,
# GR distances (0–200) dominate NPHI distances (0–0.5)
# and GR effectively controls all clustering.
```

■ **Important:** If you remove the StandardScaler, KMeans will cluster entirely on GR and RHOB (largest numerical ranges) while ignoring NPHI and SW. You will get clusters that look like GR-thresholded shale/sand, not genuine multi-curve facies.

9.4 labels_to_zones() — Why the 0.5m Minimum Thickness

```
if (base - current_top) >= min_thickness_m:
    zones.append(ZoneInterval(top=current_top, base=base, ...))
# min_thickness_m=0.5 filters out noise spikes
# A single depth sample at 0.1m sampling = 0.1m 'zone'
# These are tool noise, not real geological boundaries
```

Explanation: At 0.1m (4-inch) sampling, a single misclassified point creates a 0.1m 'zone'. On a geological log display, this appears as a noise spike that confuses the geologist. The 0.5m minimum corresponds to the approximate vertical resolution of the GR and density tools (they average over ~0.6m), so sub-0.5m features are below tool resolution and should be treated as noise.

10. analysis/reservoir.py — Reservoir Characterisation

10.1 CutoffSet — Why Four Cutoffs

```
class CutoffSet:
    phi_min  : float = 0.08   # 8% porosity minimum
    sw_max   : float = 0.60   # 60% water saturation maximum
    vsh_max  : float = 0.35   # 35% shale volume maximum
    perm_min : float = 0.1    # 0.1 mD permeability minimum
```

Explanation: The four cutoffs define 'net pay' — intervals that can actually produce. phi_min: below 8% porosity, there is not enough pore space to contain economically significant hydrocarbons. sw_max: above 60% water, the well would produce mostly water. vsh_max: above 35% clay, permeability is too low and clay swelling would damage the well. perm_min: below 0.1 mD, the formation cannot flow hydrocarbons to the wellbore at economic rates without stimulation.

10.2 compute_net_pay() — Why np.gradient Instead of Fixed Step

```
depths = df[depth_col].values
dz = np.abs(np.gradient(depths))
# np.gradient computes local spacing between depth samples
# WHY not multiply sample count by fixed step?
# LAS files sometimes have variable sampling:
#   0.1m in reservoir intervals, 0.5m in shale
# np.gradient handles this correctly; fixed step does not.
return float(np.sum(dz[mask.values]))
```

Explanation: np.gradient uses central differences: dz[i] = (depth[i+1] - depth[i-1]) / 2. For a uniformly sampled well this equals the constant step. For a variable-step well it gives the correct local thickness for each sample. Using the wrong method on a variable-step well would give net pay values that are systematically wrong by the ratio of average step to actual step.

10.3 compute_net_gross() — Gross vs Net vs N/G

```
gross  = float(np.sum(dz))            # Total interval thickness
mask   = apply_cutoffs(df, cutoffs)   # Which samples are net reservoir
net    = float(np.sum(dz[mask]))       # Thickness meeting all cutoffs
ng     = net / gross                   # Net/Gross ratio (0–1)

return {
    'gross_m'  : gross,
    'net_m'    : net,
    'net_gross': ng,
    'gross_ft' : gross * M_TO_FT,     # Also returned in feet
    'net_ft'   : net   * M_TO_FT,
}
```

Explanation: Gross is the entire logged interval regardless of quality. Net is the subset meeting reservoir quality cutoffs. N/G = 0.4 means 40% of the drilled formation is reservoir quality. N/G is a key input to reservoir simulation and material balance calculations. A Libyan carbonate field might have N/G = 0.3–0.5; a Murzuq Ordovician sandstone might have N/G = 0.6–0.8.

10.5 stoiip_bbl() — Every Constant in the Formula

```
def stoiip_bbl(area_acres, net_pay_ft, phi, sw, bo=1.2):
    return (7758 * area_acres * net_pay_ft * phi * (1 - sw)) / bo
#          ↑
# 7758 = number of barrels in one acre-foot
```

```
# 1 acre = 43,560 sq ft, 1 ft depth = 43,560 cu ft
# 1 cu ft = 0.178107 bbl, so 43,560 × 0.178107 = 7758 bbl/acre-ft
#
# phi × (1 - sw) = hydrocarbon pore volume fraction
# bo = oil formation volume factor (reservoir bbl / surface STB)
# Dividing by bo converts reservoir volumes to stock-tank barrels
```

11. analysis/statistics.py

11.1 descriptive_stats() — Why Both P10 and Mean

```
return StatisticalResult(
    mean     = float(np.mean(clean)),
    p10      = float(np.percentile(clean, 10)),
    p50      = float(np.percentile(clean, 50)),
    p90      = float(np.percentile(clean, 90)),
    skewness = float(stats.skew(clean)),
)
```

Explanation: Permeability is log-normally distributed — a few very high-permeability streaks pull the arithmetic mean far above the median. For permeability, P50 is far more useful than mean. For porosity (more normally distributed), mean ≈ P50. Skewness quantifies this: skewness > 1.0 for permeability is normal; skewness > 1.0 for porosity suggests a bimodal (dual-porosity) system.

11.2 normality_test() — Why Three Methods

Shapiro-Wilk	Most statistically powerful test for normality. Sensitive to sample size — truncated at 5000 samples to keep it meaningful. p > 0.05 = cannot reject normality. Used as the default.
Kolmogorov-Smirnov	Tests whether the empirical CDF matches a theoretical normal CDF. Less powerful than Shapiro-Wilk but works on any sample size. Used as a cross-check.
D'Agostino-Pearson	Tests for excess skewness and kurtosis separately, then combines them. Useful when the distribution looks almost normal but has a heavy tail (kurtosis) — common in resistivity logs.

11.4 monte_carlo_porosity() — Why This Matters

```
samples = np.random.normal(phi_mean, phi_std, n_samples)
samples = np.clip(samples, 0, 1)   # Physical constraint
# P10 = pessimistic case, P50 = base case, P90 = optimistic
```

Explanation: Deterministic STOIIP uses one porosity value. But porosity has uncertainty — tool calibration error ±0.5%, borehole conditions, lithology variation. Monte Carlo propagates that uncertainty through the STOIIP formula to give a range: P10 STOIIP might be 50 MMSTB, P90 might be 120 MMSTB. This range is what reserves bookings require under SPE-PRMS standards.

12. analysis/log_correlation.py

12.1 resample_to_common_depth() — Why Interpolation Is Needed

```
start = max(np.nanmin(da), np.nanmin(db))
stop  = min(np.nanmax(da), np.nanmax(db))
grid  = np.arange(start, stop + step/2, step)
# Creates a common depth axis covering only the overlap
# WHY step/2 in stop: prevents off-by-one at exactly the last depth
```

Explanation: Two wells drilled to different depths have different depth arrays. Well A might start at 2000m and go to 3500m; Well B from 1800m to 4000m. The overlap is 2000–3500m. Correlation only makes sense in the overlap — correlating data that one well doesn't have would be extrapolation, not correlation. np.interp then resamples both wells to the same grid, making sample-to-sample comparison valid.

12.2 Cross-Correlation for Depth Lag

```
xcorr = np.correlate(norm_a, norm_b, mode='full')
lag_idx = int(np.nanargmax(xcorr))  # Find peak of cross-correlation
lag_m = (lag_idx - (len(norm_a) - 1)) * actual_step
# Positive lag: Well A patterns arrive SHALLOWER than Well B
# Negative lag: Well A patterns arrive DEEPER than Well B
# Interpretation: structural dip between wells
```

Explanation: Cross-correlation slides one signal over another and measures the match at each offset. The offset at maximum match is the depth shift needed to align the two wells. A lag of +50m between two wells 5km apart implies a structural dip of arctan(50/5000) = 0.57° — a gentle dip typical of Sirte Basin horst blocks.

13. analysis/petro_summaries.py

13.1 Quality Thresholds — Calibrated to Libyan Data

Porosity thresholds	Poor <5%, Fair 5–10%, Good 10–18%, Excellent >18%. These are lower than North Sea standards because Sirte carbonates with 8–10% matrix porosity plus fracture permeability can still be commercial.
Permeability thresholds	Poor <1 mD, Fair 1–10 mD, Good 10–100 mD, Excellent >100 mD. Standard industry classification. One millidarcy allows approximately 1 bbl/day per foot of net pay under typical reservoir conditions.
Geometric mean for permeability	np.exp(np.log(k).mean()) — the log-space average equals the geometric mean. Used because permeability is log-normally distributed. A well with one 1000 mD streak and nine 1 mD intervals has arithmetic mean = 100 mD but geometric mean = 3.2 mD. The geometric mean better represents the bulk flow capacity.

13.3 interpret_dst() — Skin Factor Decision Logic

```
skin = test.get('skin_factor')
if skin is not None:
    cond = 'damaged'    if skin > 5    else \
           'stimulated' if skin < -2   else \
           'undamaged'
    lines.append(f'  Skin: {skin:.1f}  ({cond})')
```

Explanation: The skin factor (S) from pressure transient analysis measures wellbore damage. S > 5: the formation near the wellbore is damaged (mud invasion, clay swelling, fines migration) — acid stimulation is recommended. S < -2: the well is artificially stimulated (natural fractures or acid fracturing created a negative skin). S ≈ 0: undamaged, unfractured formation. These thresholds (5 and -2) are standard industry guidance from Craft & Hawkins.

14. ml/model_comparison.py — Machine Learning

Why three algorithms and how they differ

14.1 Why These Three Algorithms?

Machine learning in petrophysics serves a specific purpose: predicting a curve that was not measured (or was lost due to tool failure) from other curves that were measured. The three algorithms cover the spectrum from simplest to most powerful:

- **Linear Regression:** The baseline. If the relationship between logs and the target is approximately linear (which it often is for porosity vs RHOB), LR will capture it efficiently with no risk of overfitting. Its coefficients are directly interpretable: coefficient of RHOB = -3.5 means porosity decreases 3.5 units per unit increase in RHOB.
- **Random Forest:** Builds many decision trees on random subsets of data and features (bootstrap aggregation). Handles non-linear relationships and interactions between features without any tuning. The feature_importances_ attribute shows which log curves were most predictive — directly useful for petrophysical understanding.
- **XGBoost:** Sequential boosting — each tree corrects the errors of the previous ones. Generally the highest accuracy algorithm for tabular data. More prone to overfitting if not tuned. We compare all three so the geologist can see whether the extra complexity is justified by a meaningful R^2 improvement.

14.2 train_test_split() — Why 80/20 and Not Cross-Validation

```
self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
    self.X, self.y, test_size=test_size, random_state=random_state
)
```

Explanation: For well log data (depth series), we do NOT use cross-validation. Cross-validation would randomly assign training and test samples from throughout the well — but adjacent depth samples are highly autocorrelated (the GR at 2000m is almost identical to GR at 2001m). A model trained on 2001m and tested on 2000m has essentially seen the test data. We use a simple holdout split instead, keeping the evaluation honest.

14.3 Evaluation Metrics — What Each Means

MAE (Mean Absolute Error)	Average |predicted - actual|. Same units as the target. MAE = 0.02 on PHIE means the model is on average 2% porosity units wrong. Robust to outliers — one large error does not dominate.
RMSE (Root Mean Squared Error)	Like MAE but squares errors first, then square roots. RMSE penalises large errors more than small ones. If RMSE >> MAE, the model makes occasional very large errors. If RMSE ≈ MAE, errors are uniformly distributed.
R^2 Score (R-squared)	Fraction of variance explained: 1.0 = perfect, 0.0 = predicting the mean, negative = worse than predicting the mean. $R^2 > 0.85$ is generally considered good for well log prediction. $R^2 < 0.5$ means the selected features cannot predict this target.

14.7 serialize_model() — Why Pickle + Base64

```
def serialize_model(model) -> str:
    pickled = pickle.dumps(model)              # Convert to bytes
    return base64.b64encode(pickled).decode()  # Bytes → ASCII string

# WHY base64: SQLite TEXT columns store text, not binary.
# pickle creates binary bytes that contain NULL bytes and
```

```
# characters that would break TEXT storage.
# base64 encodes 3 bytes as 4 ASCII characters — safe for any text field.
```

15. ml/trend_analysis.py — Depth Trends

15.1 Sequential Split — Why Not Random

```
split_idx = int(len(self.X) * (1 - test_size))
self.X_train = self.X[:split_idx]   # First 80% by depth
self.X_test  = self.X[split_idx:]   # Last 20% by depth (deepest samples)
```

Explanation: For depth trend analysis, depth IS the only input feature. A random split would give the model test points at depths it has already 'seen' during training (e.g. training on 2000–2500m and 2600–3000m, testing on 2500–2600m). Sequential split evaluates whether the trend found in the shallow part of the well correctly predicts the deep part — a genuine test of predictive power.

15.2 fit_linear_regression() — What the Slope Means

```
slope = float(self.lr_model.coef_[0])
intercept = float(self.lr_model.intercept_)
result['equation'] = f'y = {slope:.6f}*depth + {intercept:.2f}'
```

Explanation: For porosity vs depth: slope = –0.000050 means porosity decreases 0.005% per metre of depth. Over 1000m, that is 5% total compaction — consistent with Athy's compaction law for carbonates. A positive slope for GR would indicate increasing shale content with depth — possibly approaching a shale-dominated formation. A slope near zero means no depth trend exists.

16. ai/gemini_interpreter.py — AI Integration

16.1 _auto_select_model() — Why Dynamic Detection

```
def _auto_select_model(self) -> str:
    try:
        available = {m.name for m in genai.list_models()
                     if 'generateContent' in m.supported_generation_methods}
        for candidate in self._MODEL_CANDIDATES:  # Try priority list
            if candidate in available:
                return candidate
    except Exception:
        pass   # API call failed — use hardcoded fallback
    return 'models/gemini-1.5-flash'  # Last resort
```

Explanation: Google changed their model naming convention between API versions — 'gemini-1.5-flash' became 'models/gemini-1.5-flash'. Hardcoding either name broke every time Google made a change. Dynamic detection calls the live API list of available models and picks the best available one from a priority list. If the API call itself fails (network issue), the exception is caught and a reasonable fallback is returned — the system never crashes due to a model name.

16.2 _SYSTEM_PROMPT — Libya-Specific Calibration

The system prompt is the invisible instruction that preconditions the AI's entire response style before the user's actual question. Without it, Gemini would respond as a general geologist using global examples. With it, Gemini responds as a 30-year Libyan petroleum expert using Intisar, Sarir, Acacus, and Mamuniyat as formation analogues.

The five numbered instructions in the system prompt encode hard-won expertise:

- **Reference Libyan formation names:** Instead of 'typical carbonate reservoir', Gemini will say 'similar to Intisar D reef complex in the central Sirte Basin'.
- **Apply North African structural context:** Sirte Basin formed as a Cretaceous rift. Tethyan margin influence, inversion tectonics during the Eocene, and basement highs controlling trap geometry are all Libya-specific features.
- **NOC reporting conventions:** The National Oil Corporation has specific formats for exploration reports. Using these conventions makes the AI output directly useful for regulatory submissions.
- **Reservoir quality classes:** The specific thresholds (Excellent ϕ>20%, Good ϕ>12%, etc.) are calibrated to Libyan carbonates — lower than North Sea standards because fractured carbonates are commercial at lower matrix porosity.
- **Libya-specific risks:** Diagenesis in Sirte carbonates, overpressure in deep Ghadames, compartmentalisation in Murzuq Ordovician are all basin-specific risks that a generic AI would not mention without this guidance.

17. database/db.py — SQLite Schema Design

17.1 Why SQLite and SQLAlchemy ORM

SQLite stores the entire database in a single file (~/.project_qle/database.db). No server process, no installation, no network port, no administrator required. The file travels with the project and can be backed up by copying it. For a single-team platform with one active writer at a time (Streamlit single-user), SQLite's limitations (no concurrent writes) are irrelevant.

SQLAlchemy ORM (Object-Relational Mapper) lets us work with database records as Python objects. Instead of writing raw SQL, we write Python. The ORM also makes the code database-agnostic: migrating to PostgreSQL for a multi-user deployment would require changing only the connection string from sqlite:// to postgresql://.

17.2 Relationship Design — Why Cascade Delete

```
class Project(Base):
    wells = relationship('Well', back_populates='project',
                         cascade='all, delete-orphan')
# cascade='all, delete-orphan' means:
# When a Project is deleted, ALL its Wells are also deleted.
# Without this, deleting a project would leave orphan wells
# with project_id pointing to a non-existent project.
```

Explanation: The cascade ensures referential integrity without requiring the application code to manually clean up child records. The relationship also allows Python-level navigation: project.wells gives all wells for a project, well.project gives the parent project — no explicit SQL JOINs required in application code.

18. database/auth.py — Access Control System

18.1 Why SHA-256 Key Hashing

```
def _hash(key: str) -> str:
    return hashlib.sha256(key.encode('utf-8')).hexdigest()
# ONE-WAY: given hash → cannot recover key
# DETERMINISTIC: same key always → same hash
# 64-character hex string stored in database
# Even if database is stolen: hashes reveal nothing
```

Explanation: If we stored access keys in plain text and the database were stolen, the attacker immediately has all keys. With SHA-256 hashes, the attacker sees 64-character hex strings and cannot reverse them to the original keys. SHA-256 is a one-way cryptographic function: computing SHA-256(key) is fast, but finding a key that produces a given hash requires testing ~2^256 possibilities — computationally infeasible with any hardware that will ever exist.

18.2 Key Generation with the secrets Module

```
import secrets
def _generate_key(length: int = 20) -> str:
    alphabet = string.ascii_letters + string.digits  # 62 characters
    return 'QLE-' + ''.join(secrets.choice(alphabet) for _ in range(length))
# secrets.choice uses cryptographically secure random numbers
# NOT random.choice — regular random is predictable if the seed is known
```

Explanation: Python's random module uses a Mersenne Twister — a pseudorandom generator seeded from the current time. An attacker who knows approximately when a key was generated can reproduce the sequence and guess the key. secrets.choice uses the OS's cryptographic entropy source (e.g. /dev/urandom on Linux) which is truly unpredictable. For security-critical key generation, always use secrets, never random.

18.3 init_auth() — First-Run Bootstrap

```
owner = session.query(User).filter_by(role='owner').first()
if owner is None:   # First run only
    master_key = os.environ.get('QLE_MASTER_KEY') or _generate_key(24)
    # Write key to file on disk:
    with open(key_file, 'w') as f:
        f.write(f'PROJECT_QLE OWNER KEY\n{master_key}\n')
    print(f'FIRST RUN — key saved to: {key_file}')
```

Explanation: The check 'if owner is None' ensures key generation happens only once. On every subsequent launch, the owner already exists and this block is skipped. The key is written to a file rather than only printed, because Docker containers or cloud deployments may not have a human reading the console output. The file path ~/.project_qle/owner_key.txt is in the user's home directory which is outside the project source tree and will not be accidentally committed to version control.

19. pipeline.py — QLEPipeline Orchestrator

19.1 The Fluent Builder Pattern

```
def add_las(self, path) -> 'QLEPipeline':   # Returns self
    well = parse_las(path)
    self._wells.append(well)
    return self   # ← This makes method chaining possible

# Usage:
pipe = QLEPipeline('Sirte Block 47')
       .add_las('WELL_A.las')
       .add_las('WELL_B.las')
       .add_las('WELL_C.las')
```

Explanation: Returning self from add_las() and add_well() enables method chaining — a style that reads like English: 'create a pipeline, add well A, add well B, run'. Each method call returns the same pipeline object, so the next call operates on the same object. This is called the Fluent Interface or Builder Pattern.

19.2 The Five-Step Order — Why This Sequence

Step 1: Petrophysics	Must be first — all subsequent steps depend on PHIE, SW, VSHALE columns being present in well.df. If petrophysics fails for a well, that well is skipped in all subsequent steps but the others continue.
Step 2: Facies + Reservoir	Facies classification needs PHIE, SW, GR, RHOB (computed in Step 1). Reservoir summary needs facies zones from facies classification. These two are combined because reservoir.build_summary() inputs facies zones.
Step 3: Statistics	Statistics needs the enriched DataFrame from Step 1. Run after petrophysics so that derived curves (PHIE, SW) are included in the statistics.
Step 4: Correlation	Cross-well correlation uses get_depth() and get_curve() — works on any well regardless of whether petrophysics succeeded. Run last so the correlation pairs can include all loaded wells.
Step 5: AI	Last because it needs all previous results as input: reservoir summaries (Step 2), statistics (Step 3). The AI summarises everything — so everything must be computed first.

19.3 Error Handling — Why Per-Well try/except

```
for well in self._wells:
    try:
        df = PetrophysicsEngine(well, basin=self.basin).run()
    except Exception as exc:
        msg = f'Petrophys error ({well.header.well_name}): {exc}'
        report.warnings.append(msg)   # Record, not crash
        logger.error(msg)
        # Continue to next well ← this is the key behaviour
```

Explanation: In field operations, data is routinely incomplete. A well might have GR and RHOB but no sonic log. Another might have depth in a non-standard column. A third might have corrupted resistivity data. If any of these caused an exception that was not caught, the entire pipeline would abort, losing results for all other wells. Per-well try/except means one bad well produces a warning; the other five wells complete successfully.

20. app.py — Streamlit UI Deep Dive

~1900 lines — the most user-visible code

20.1 Why One File vs Many Pages

Streamlit re-runs the entire script on every user interaction. A multi-page structure (separate .py files in a pages/ folder) means shared state between pages requires st.session_state — which app.py also uses. The single-file approach allows shared helper functions (make_log_plot, _gb, _mc) to be defined once and used on every page without import overhead. For a project at this scale (18 pages, ~1900 lines), a single file is manageable. For 50+ pages, the pages/ directory structure would be appropriate.

20.2 Access Control Wall — How st.stop() Works

```
if not _check_access():
    _login_wall()   # Shows the login form

def _login_wall():
    st.markdown('## ■ Access Required')
    key_input = st.text_input('Access Key', type='password')
    if st.button('Enter'):
        user = authenticate(key_input)
        if user:
            st.session_state['authenticated_user'] = {...}
            st.rerun()   # Re-run the entire script — now authenticated
        else:
            st.error('Invalid key')
    st.stop()   # ← Halts all further script execution
```

Explanation: st.stop() is a Streamlit function that immediately halts script execution at that point. Everything below _login_wall() — all 18 pages, all data — is never executed. This is not just hiding elements with CSS visibility:hidden (which a user could bypass with browser developer tools). st.stop() means the Python code for the rest of the app never runs.

20.3 make_log_plot() — New Design Decisions

```
# CURVE NAME + SCALE AT THE TOP:
ax.set_title(f'{curve}\n({ul})  {scale_txt}',
             color=color, fontsize=7, fontweight='bold',
             pad=3, loc='center')
ax.set_xlabel('')   # Clear the bottom label

# GRID ON BOTH AXES:
ax.grid(which='major', axis='both',
        color=DARK_GRID, lw=0.5, alpha=0.7, linestyle='--')
ax.grid(which='minor', axis='x',
        color=DARK_GRID, lw=0.3, alpha=0.4, linestyle=':')
```

Explanation: Standard well log displays (like Techlog, IP, Petrel) always put the curve name at the TOP of the track. This is the industry convention because the depth axis runs vertically and the track width is narrow — a bottom label is harder to read when the track is only ~2.5 inches wide. The scale (min–max values) is also shown at the top so the geologist can immediately calibrate their eye.

Explanation: The horizontal gridlines (axis='y') make it easy to read exact depths by following a horizontal line across all tracks. The vertical gridlines (axis='x') mark the value scale divisions on each track. Both are drawn as dashed lines with low opacity (alpha=0.7) so they guide the eye without obscuring the log curve.

20.4 make_comparison_plot() — N-Well Architecture

```
total_cols = n_wells * len(avail)   # N wells × M curves = N×M subplots
fig, all_ax = plt.subplots(1, total_cols, sharey=True)
# sharey=True: all tracks share ONE depth axis
# → formations at the same depth align horizontally across all wells

for wi, (well, df, ...) in enumerate(zip(wells_list, ...)):
    start  = wi * len(avail)          # First column for this well
    w_axes = all_ax[start: start + len(avail)]  # This well's tracks
```

Explanation: The key architectural decision is sharey=True — all N×M subplots share the same y-axis. This means if Well A shows a carbonate at 7500 ft and Well B shows the same carbonate at 7450 ft (50 ft structural dip), both appear at their correct depths and the 50 ft offset is immediately visible. This is how professional correlation software works: the depth axis is shared and depth shifts between wells reveal stratigraphy and structure.

20.5 _draw_track() — Centralised Track Drawing

```
def _draw_track(ax, vals, depth, curve, color):
    # Called by BOTH make_log_plot AND make_comparison_plot
    # Ensures 100% consistent track rendering across all plot types
    # Any change here applies everywhere — no duplicate code
```

Explanation: DRY principle (Don't Repeat Yourself). Before _draw_track() existed, the fill logic, grid settings, title placement, and scale calculation were duplicated between the single-well plot and the comparison plot. A bug fix in one would not be applied to the other. Centralising in _draw_track() means both functions automatically benefit from any improvement.

20.6 _d2ft() and the use_feet Design

```
def _d2ft(v):   # Depth metres → feet for display
    from project_QLE.units import M_TO_FT
    if v is None: return None
    return v * M_TO_FT   # Works on scalars AND numpy arrays

# All plot functions accept use_feet=True parameter:
def make_log_plot(..., use_feet=True):
    df_disp = _depth_display(df, depth_col) if use_feet else df.copy()
```

Explanation: The use_feet parameter preserves the ability to display in metric for users who prefer it (e.g. European service companies) while defaulting to feet for Waha Oil Company operations. It is a display-only flag — all internal calculations still use metres. The conversion is applied once at display time, not at calculation time.

20.7 Session State — The Memory of a Stateless App

```
def ss(key, default=None):
    if key not in st.session_state:
        st.session_state[key] = default  # Set only on first access
    return st.session_state[key]

wells = ss('wells', [])   # Returns [] on first load, list of wells thereafter
```

Explanation: Streamlit re-runs the entire Python script on every button click, slider move, or page navigation. Without session_state, all computed data (loaded wells, petrophysics results, facies zones) would be lost on every interaction. st.session_state is a dictionary that persists across reruns for the same browser session. The ss() helper initialises keys with defaults on first access to prevent KeyError on the initial page load before any data is uploaded.